

☐ MOZIEST FOGLALÁSI RENDSZER

JavaScript gyakorlófeladat — DOM • tömbök • objektumok • radio gombok • querySelector
• localStorage • időzítő

Feladat	Készíts egy webalkalmazást, amely egy moziest foglalásait kezeli. A felhasználó megadhatja a nevét, kiválaszthat egy filmet, megadhatja a jegyek számát, valamint rádiógombokkal kiválaszthatja az ülőhely típusát.
Adatkezelés	A foglalások kerüljenek a böngésző localStorage-ába, így újratöltés után is maradjanak meg. A mentett foglalások táblázatban jelenjenek meg.
Időkorlát	A foglalás elkészítésére 60 másodperc áll rendelkezésre. Az idő lejártá után új foglalás ne legyen létrehozható.

1. ADATOK

Hozz létre egy filmek nevű tömböt! A tömbben az alábbi filmek szerepeljenek:

- Interstellar
- Inception
- The Dark Knight
- Spider-Man: Into the Spider-Verse

A helytípusokhoz tartozó jegyárak:

Helytípus	Ár / jegy
Normal	2500 Ft
VIP	4000 Ft
Premium	5500 Ft

A végső ár = jegyek száma × a választott helytípus jegyára.

Egy mentett foglalás például az alábbi adatokat tartalmazza:

```
{
  nev: "Kiss Péter",
  film: "Interstellar",
  jegyek: 2,
  hely: "VIP",
  ar: 8000
}
```

2. FELHASZNÁLÓI FELÜLET

Az elkészített index.html már tartalmazza a szükséges HTML-elemeket. Ezeket használd:

Azonosító / név	Feladat
#nev	név megadása
#film	film kiválasztása
#jegyek	jegyek számának megadása
name="hely"	három rádiógomb az ülőhely típusához
#foglalasForm	úrlap
#foglalasGomb	foglalás gomb
#uzenet	hiba- és sikerüzenetek
#foglalasok tbody	foglalások táblázata
#idozito	időzítő megjelenítése
#foglalasokSzama	foglalások száma
#jegyekSzama	eladott jegyek száma
#ujrainditasGomb	új foglalási időszak

3. ŰRLAP ELLENŐRZÉSE

Az űrlap elküldésekor:

- a név nem lehet üres;
- kötelező legyen filmet választani;
- a jegyek száma legalább 1 legyen;
- legyen kiválasztva ülőhelytípus;
- hibás adat esetén jelenjen meg hibaüzenet;
- sikeres foglalás esetén készüljön egy objektum a megadott adatokból.

SEGÍTSÉG — RADIO GOMB

A kiválasztott rádiógomb értékét a `querySelector` segítségével keresd meg.

```
document.querySelector('input[name="..."]:checked')
```

Ha nincs kiválasztva rádiógomb, a `querySelector` eredménye null lehet.

4. ÁRSZÁMÍTÁS

Készíts egy `arSzamitas()` nevű függvényt! A függvény kapja meg a jegyek számát és a helytípus nevét, majd adja vissza a végső árat.

SEGÍTSÉG — ÁRAK TÁROLÁSA

Használhatsz egy objektumot, amelyben a helytípusokhoz tartozó árakat tárolod.

```
const arak = { Normal: ..., VIP: ..., Premium: ... }
```

5. LOCALSTORAGE

A foglалásokat foglалások néven tárold localStorage-ban, JSON formátumban.

- betöltéskor olvasd ki a korábban mentett foglалásokat;
- új foglалáskor tedd hozzá az objektumot a tömbhöz;
- mentsd vissza a tömböt localStorage-ba;
- törléskor távolítsd el a megfelelő elemet;
- törlés után frissítsd a táblázatot.

SEGÍTSÉG — JSON

A localStorage szöveges adatot tárol.

`JSON.stringify(...)` és `JSON.parse(...)` használható.

A korábbi adatok hiányára is gondolj.

6. TÁBLÁZAT

Készíts egy foglалásokLista() nevű függvényt! A függvény:

- törölje a táblázat korábbi tartalmát;
- menjen végig a foglалások tömbjén;
- minden foglалáshoz hozzon létre egy új táblázatsort;
- jelenítse meg a nevet, filmet, jegyek számát, helytípust és árat;
- minden sorhoz tartozzon törlés gomb.

A törlés gomb kattintásakor az adott foglалás kerüljön ki a tömbből, majd a módosított tömb kerüljön vissza a localStorage-ba.

7. STATISZTIKA

A statisztikában jelenjen meg:

- hány foglалás van összesen;
- hány jegyet foglaltak összesen.

A statisztikát minden hozzáadás és törlés után frissíteni kell.

SEGÍTSÉG — TÖMBÖK

A jegyek összegzéséhez használhatsz forEach() ciklust vagy más tömbbejárási módszert.

8. IDŐZÍTŐ

Az oldal indulásakor induljon egy 60 másodperces visszaszámlálás.

FOGLALÁSI IDŐ	60 mp
Lejáratkor	A foglалás elküldése ne legyen lehetséges.

Készíts egy idoFrissitese() nevű függvényt! A függvény:

- másodpercenként csökkentse az időt;
- frissítse a #idozito tartalmát;
- ha az idő eléri a 0-t, állítsa le az időzítőt;
- tiltsa le a foglалás elküldését;
- jelenítsen meg megfelelő üzenetet.

Az időzítőhöz használj setInterval()-t és clearInterval()-t.

FONTOS

Új setInterval() indítása előtt gondoskodj arról, hogy a korábbi időzítő le legyen állítva.

9. ÚJ FOGLALÁSI IDŐSZAK

Az „Új foglалási időszak” gombra kattintva:

- az idő álljon vissza 60 másodpercre;
- induljon újra az időzítő;
- a foglалási űrlap ismét legyen használható;
- az üzenet törlődjön.

10. JAVASCRIPT FÜGGVÉNYEK

arSzamitas()	A foglalás végső árának kiszámítása.
mentes()	A foglalások mentése localStorage-ba.
foglalasokLista()	A mentett foglalások megjelenítése.
torles()	A megadott foglalás eltávolítása.
frissitStatiztika()	A darabszámok frissítése.
idoFrissitese()	A visszaszámlálás kezelése.

11. KÖVETELMÉNYEK

A program külön fájlokból álljon:

index.html	A felhasználói felület.
style.css	Az oldal megjelenése.
script.js	A JavaScript program.

A JavaScript-kód használjon:

- DOM-műveleteket;
- `querySelector()` vagy `querySelectorAll()` használatát;
- eseménykezelést;
- tömböket;
- objektumokat;
- tömbbejárást;
- rádiógombokat;
- `localStorage`-t;
- `JSON.parse()` és `JSON.stringify()` használatát;
- `setInterval()` és `clearInterval()` használatát.

12. TESZTELÉS

Próbáld ki az alábbi eseteket:

- üres névvel próbálsz foglalni;
- nem választasz filmet;
- nem választasz helytípust;
- több jegyet foglalsz;
- több különböző foglalást hozol létre;
- újratöltöd az oldalt;
- törölsz egy foglalást;
- megvárod, amíg lejár a 60 másodperc;
- új foglalási időszakot indítasz;
- ellenőrzöd, hogy a `localStorage` tartalma megfelelően változik.

FONTOS

A HTML és CSS elkészítve rendelkezésre áll. A JavaScriptet önállóan készítsd el.

A cél nem az, hogy egyetlen hosszú függvényben oldd meg az egészet, hanem hogy a programot kisebb, jól elkülönített feladatokra bontsd.